System and Network Compromising Exam

Execution Summary

- You are hired to perform offline security assessment. You will be provided with an archive, containing all your tasks.
- This is a template for uploading your screenshots.
- The exam objective is to collect 6 points, each by solving specific tasks.
- All tasks are giving equal number of points.

Do not overstress or overcomplicate it. The time is enough, you can do it.

Scope

On the SoftUni exam page you will find <exam.zip> file, download it and extract it. (Depending on your AV software, sometimes it may flag, make sure to disable it or to extract the documents under isolated folder)

Appendix

This is the screenshot section. It is a good practice to explain your finding (if applicable). After the description, make sure to drop your screenshots below.

Task 1

After extracting the archive, Import the **BloodHound.zip** into **BloodHound**. Mark the user **Olivia** as owned. Can Olivia compromise any other user? If so, what is the attack vector (ACL). Can it be abuse from both Windows and Linux environment?

Description:
->After marking Olivia as "Owned" and using the 'First Degree Object Control' query revealed that Olivia has a GenericAll permission (full control) associated with Michael. This misconfiguration suggests that if Olivia's account is compromised, it might be used to influence or manipulate Michael's account. Additionally, I reviewed Olivia's group memberships, noting that she belongs to Remote Management and Domain Users (with Domain Users being further nested under Everyone, Users, and Authenticated Users). This helps explain how permissions could propagate in the environment. The key finding is the GenericAll ACL between Olivia and Michael. With GenericAll, Olivia (once controlled) can potentially modify Michael's account attributes, which may include delegating rights or setting up further exploits. Further exploration revealed that Michael possesses an outbound 'Force Change Password' right over Benjamin. This indicates a potential attack chain: starting from Olivia, leverage her control over Michael, then use Michael's right to force a password change on Benjamin. Even though Benjamin's account has limited privileges, this chain demonstrates how misconfigured permissions allow lateral movement. The exploitation of these ACL misconfigurations isn't bound to a single platform. Tools available on both Windows (PowerShell with Active Directory modules) and Linux ("ldapmodify" or "ldappasswd" on Kali Linux) can be used to perform these actions. This confirms that the attack vector is indeed abuseable from both environments.
Screenshots:

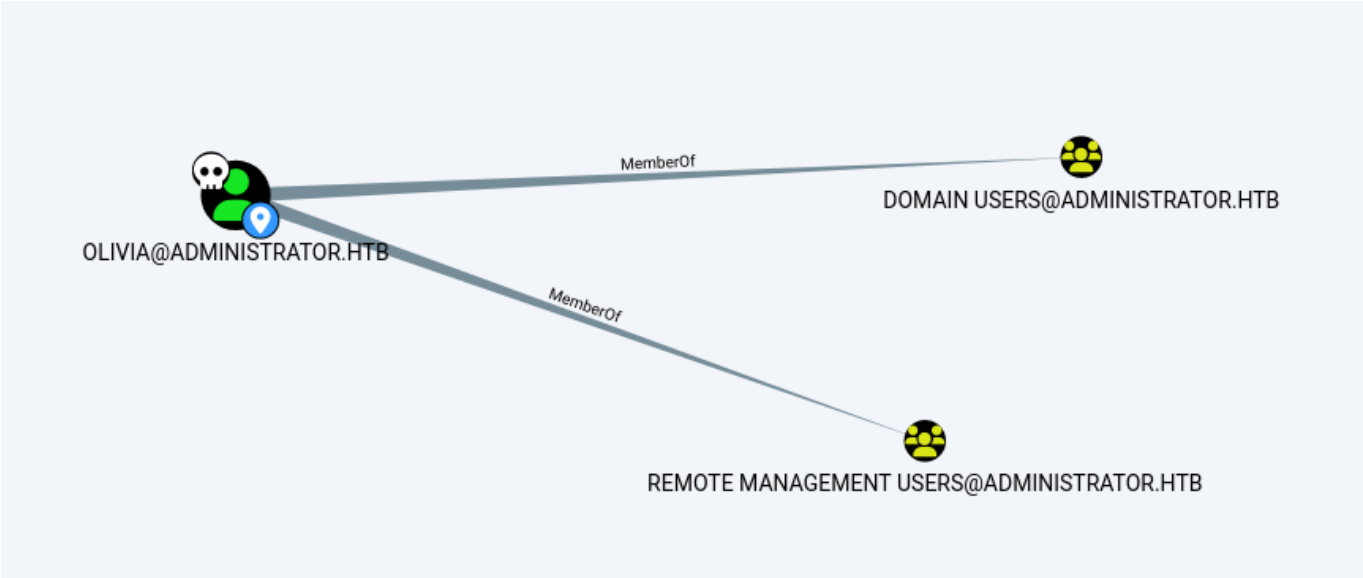
SoftUni

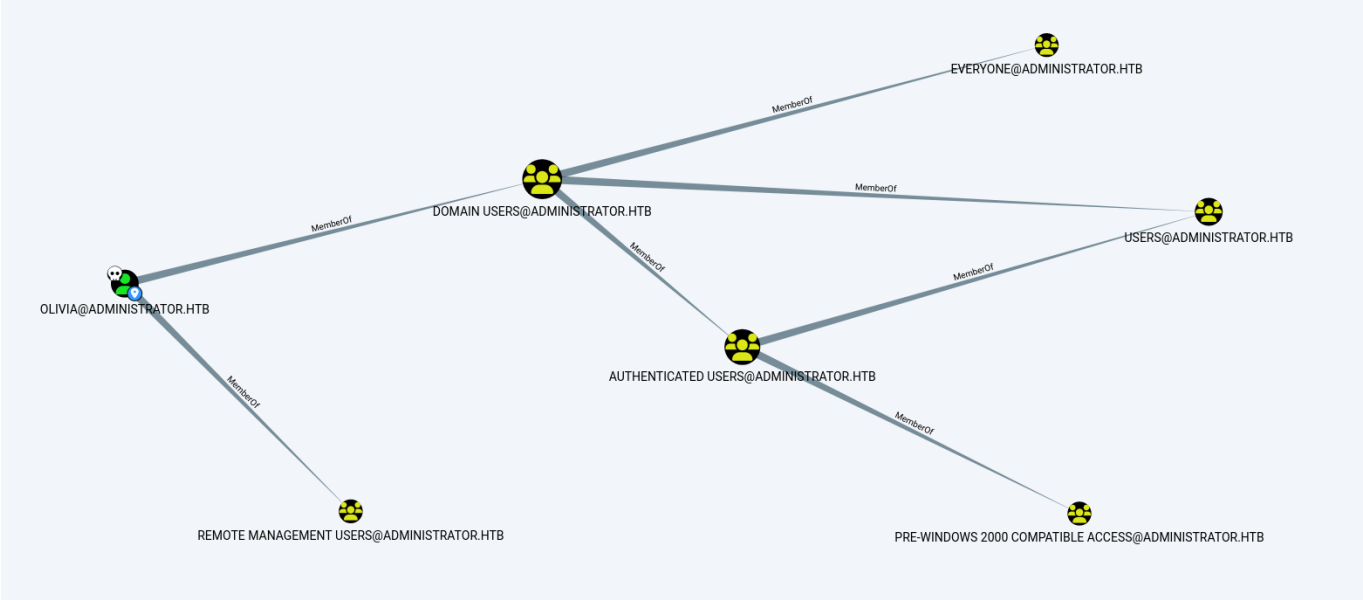
© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.

Follow us:

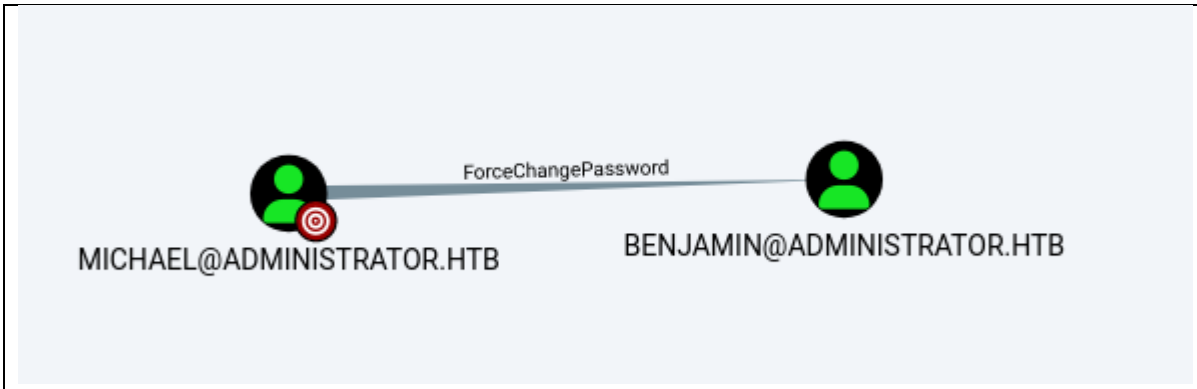
->







SoftUni
© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.
Follow us:



Task 2

Using the same BloodHound data, find user with the SID: S-1-5-21-1088858960-373806567-254189436-1110. What is the name of this user? Which groups this user is a member of? Can this user be compromised? If so, with what ACL abuse?

Description:

-> I began by searching for the SID S-1-5-21-1088858960-373806567-254189436-1110 within the BloodHound dataset. The query returned the user 'Benjamin', confirming his identity as the account associated with this SID.

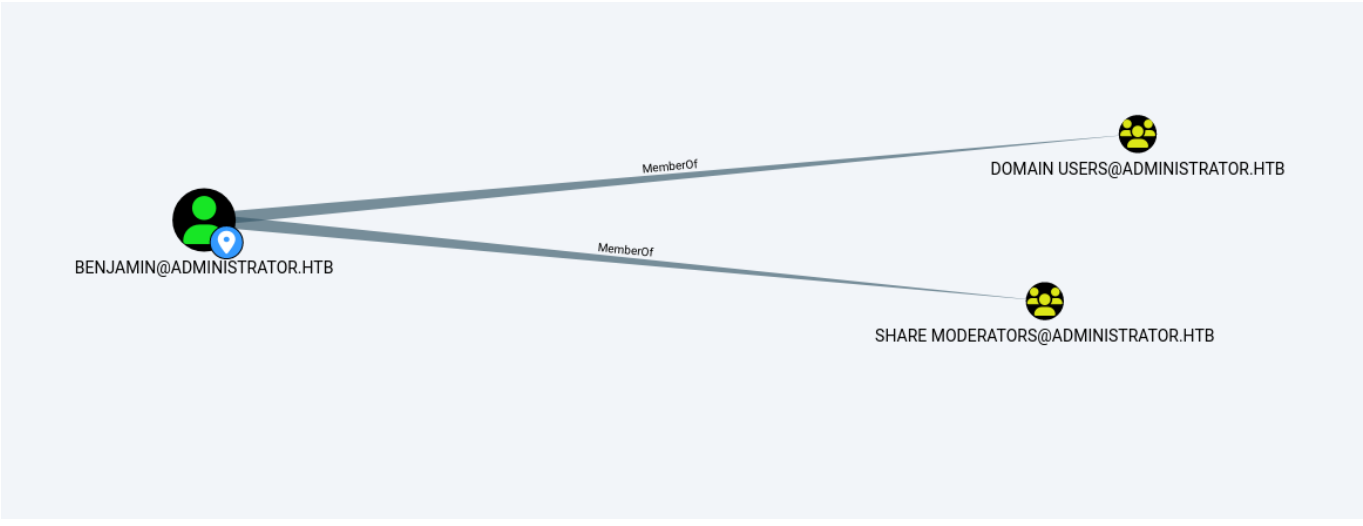
Benjamin is directly a member of the groups Domain Users and Share Moderators. This is illustrated in the first screenshot, which shows his first degree group memberships.

Further analysis of the unrolled group memberships reveals that the Domain Users group is nested within broader groups such as Everyone, Users, and Authenticated Users. Notably, Authenticated Users is also a member of PRE-Windows 2000 Compatible Access. This nesting indicates that Benjamin indirectly inherits a wide range of permissions from these larger groups. A second screenshot demonstrates this unrolled group membership.

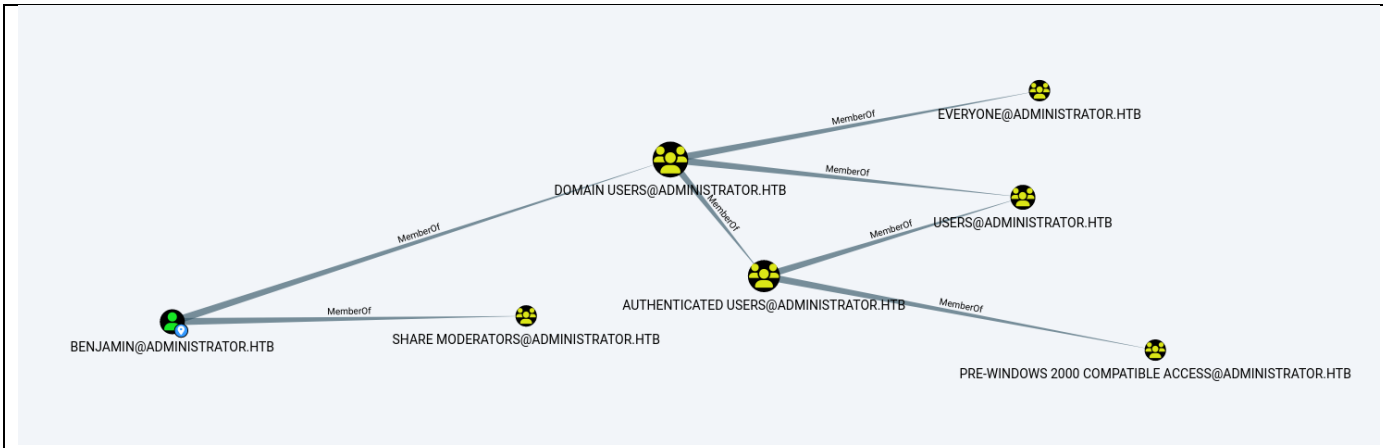
Although Benjamin's direct group memberships are limited, the environment exhibits a misconfiguration that can be exploited. From Task 1, we observed that Michael's account has an outbound 'Force Change Password' ACL over Benjamin. This means that if an attacker controls Michael's account (or can manipulate it using the GenericAll relationship from Olivia), they can force a password reset on Benjamin's account.

Screenshots:

->



© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.
Follow us:



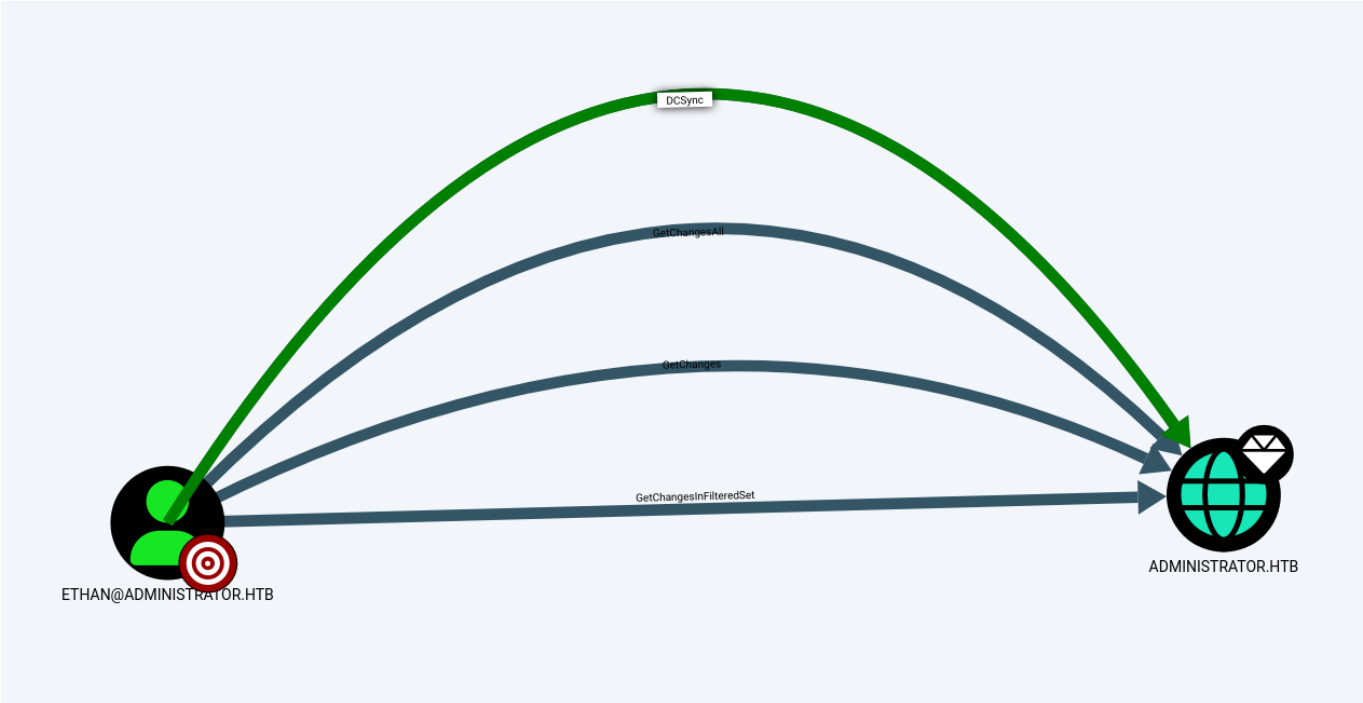
Task 3

Using the same BloodHound data, find which user can synchronize the Active Directory but its **NOT** a Domain Admin.

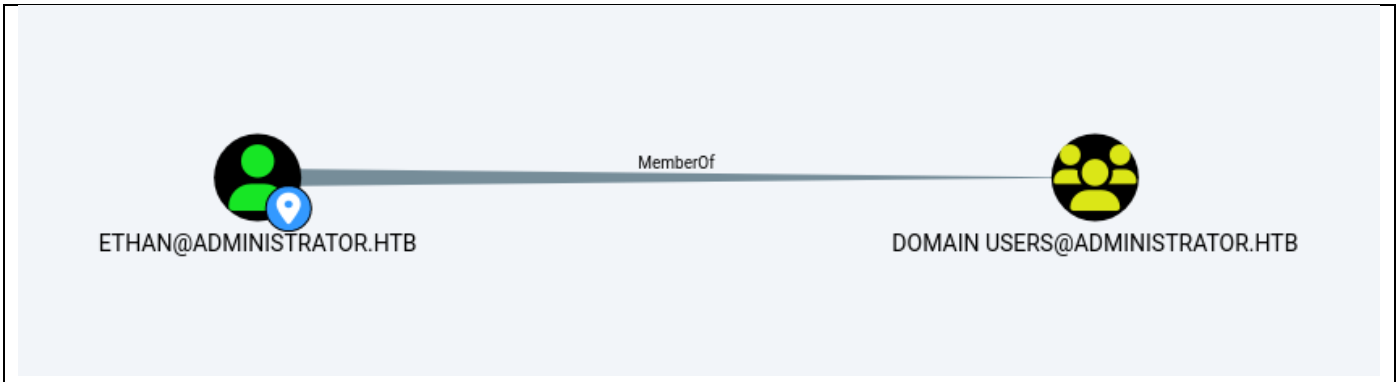
Description:

> **The Domain Admin in our environment is ADMINISTRATOR@ADMINISTRATOR so I will be ignoring it.I began by running a query in BloodHound to identify accounts with the DCSync privilege—that is, those with effective permissions such as Replicating Directory Changes or Replicating Directory Changes All on the Domain object. The results revealed that the account Ethan (displayed as ETHAN@ADMINISTRATOR) possesses these privileges. And he is part of DOMAIN USERS group so he is not a domain admin as he is not part of DOMAIN ADMINS group (like ADMINISTRATOR@ADMINISTRATOR).**

Screenshots:



© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.
Follow us:



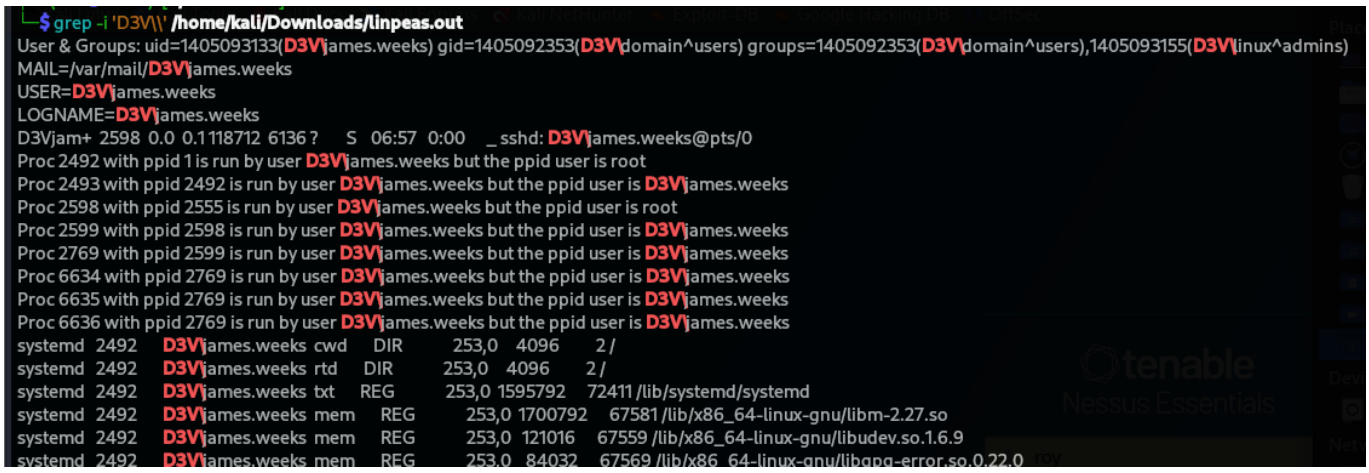
Task 4

Find and open linpeas.out file. Find the Domain users that connected to the linux machine on which the linpeas was executed.

Description:
->I used “less linpeas.out” and “grep -i 'D3V\\' linpeas.out”. This command returned multiple lines indicating session details and environment variables associated with the domain user. **The grep output clearly showed entries like:** **- User & Groups: uid=1405093133(D3V\james.weeks) gid=1405092353(D3V\domain^users)...** **- Session logs, such as: sshd: D3V\james.weeks@pts/0** **- Process details confirming that processes are running under the domain user “D3V\james.weeks”** **These details confirmed that the Linux machine was accessed by the domain user “D3V\james.weeks”**
Screenshots:
->

SoftUni
© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.
Follow us:





Task 5

Imagine you have the following C2 implant: C:\Windows\Tasks\updater.exe. Establish persistence via shortcut links, using ONLY the CLI (cmd or powershell). Provide commands and screenshots if possible. The commands must be easy to reproduce and valid.

Description:

> **Run Powershell as administrator and execute this script:**

```
# Create the WScript.Shell COM object
$WshShell = New-Object -ComObject WScript.Shell
```

© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.
Follow us:

```
# Define the path for the shortcut in the current user's Startup folder
$ShortcutPath = "$env:APPDATA\Microsoft\Windows\Start
Menu\Programs\Startup\updater.lnk"

# Create the shortcut object
$Shortcut = $WshShell.CreateShortcut($ShortcutPath)

# Set the target of the shortcut to the C2 implant executable
$Shortcut.TargetPath = "C:\Windows\Tasks\updater.exe"

# Optionally set the working directory
$Shortcut.WorkingDirectory = "C:\Windows\Tasks"

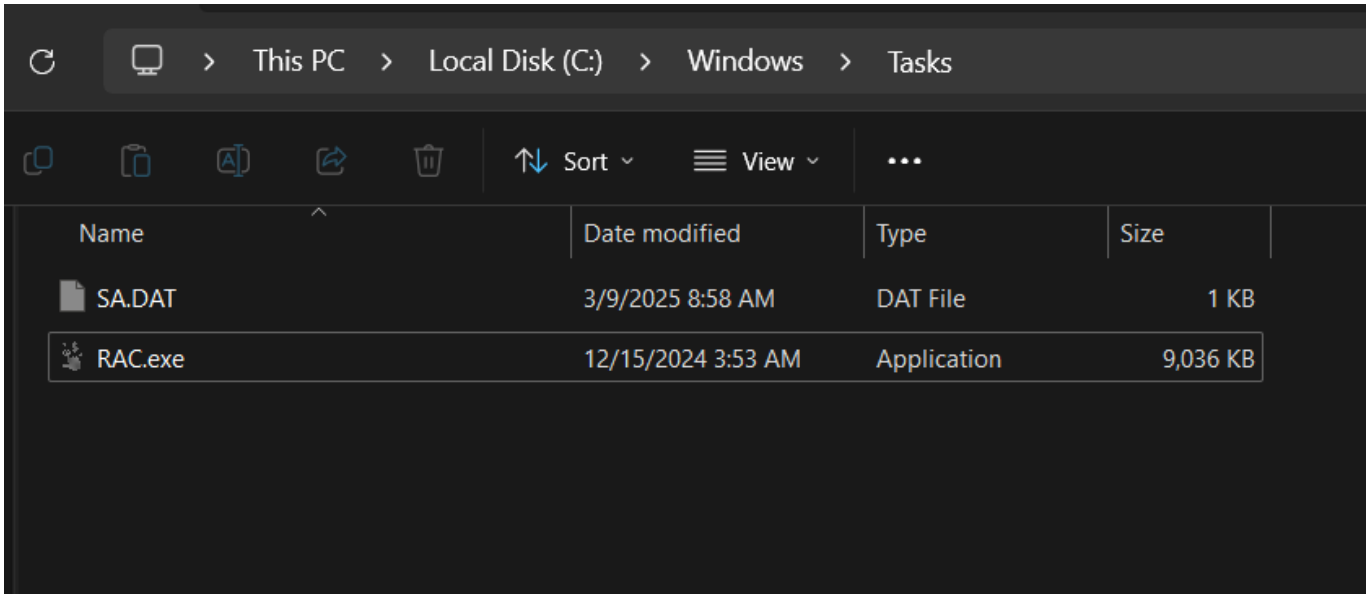
# Add a description (optional)
$Shortcut.Description = "C2 implant updater persistence shortcut"

# Save the shortcut
$Shortcut.Save()
```

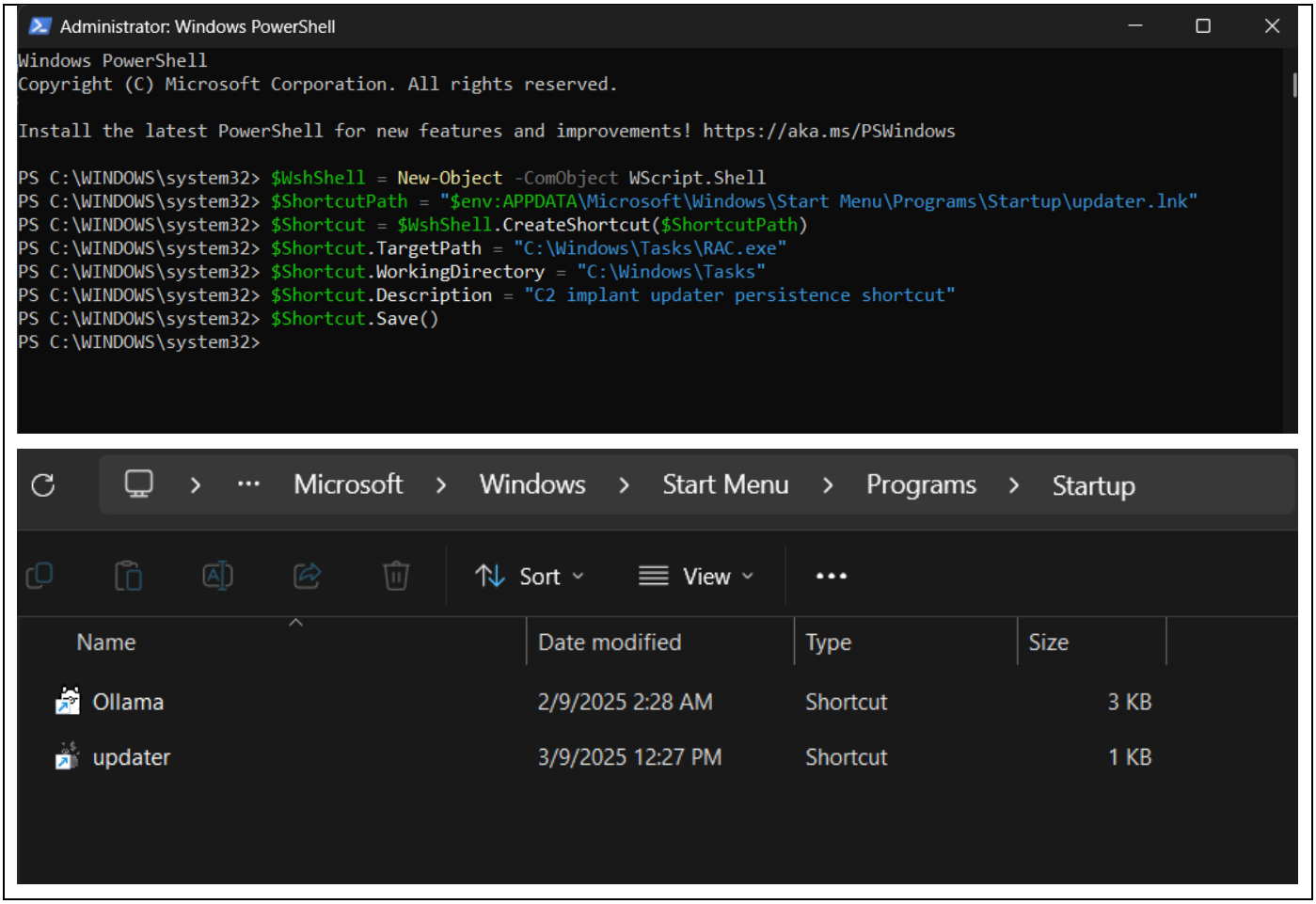
- **In my case I have my own RAC.exe (Random Auto Clicker – Auto Clicker with random delay between 2 float values). So I will use it for example in the screenshots.**

Screenshots:

->



© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.
Follow us:

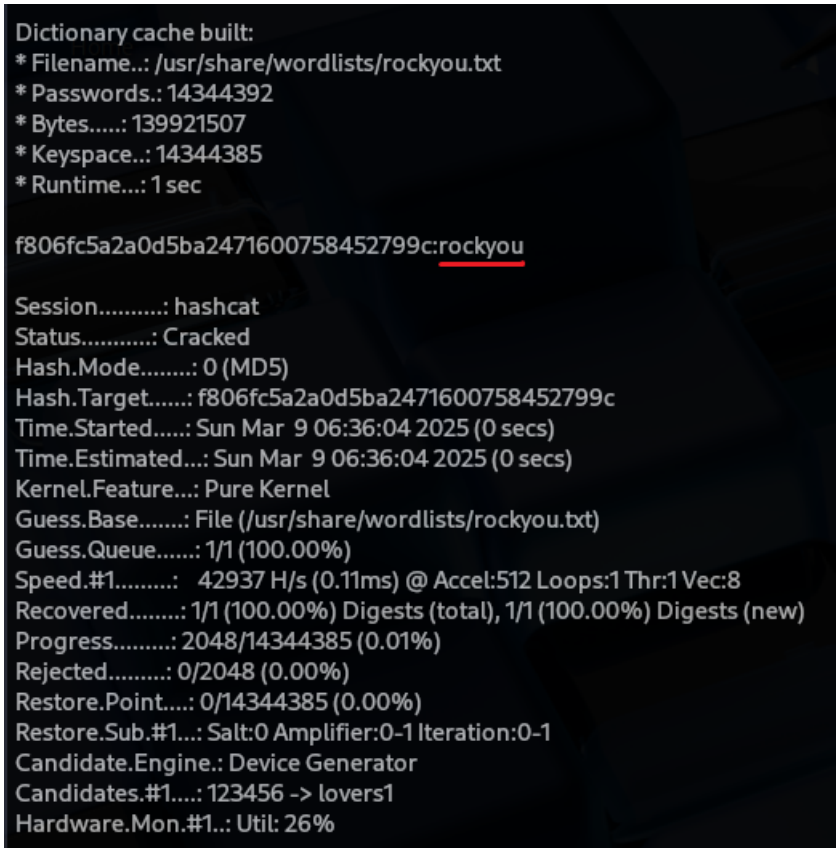


Task 6

Find the hash inside **hash.txt**. Understand its type and use **hashcat** to retrieve (crack) the original password.

Description:
> **I opened the file hash.txt and found the following hash: f806fc5a2a0d5ba2471600758452799c** > **Given that MD5 hashes are 32 hexadecimal characters, I determined that this hash is most likely an MD5 hash.** > **Since the hash type is MD5, I used hashcat's mode -m 0 (which is designated for MD5) along with a dictionary attack mode -a 0. I chose a common wordlist (rockyou.txt) to attempt cracking the hash.** > **Running this command: "`hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/rockyou.txt`"** > **After executing the command, hashcat processed the hash and the password corresponding to that hash is "rockyou"**
Screenshots:

© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.



© SoftUni – about.softuni.bg. Copyrighted document. Unauthorized copy, reproduction or use is not permitted.